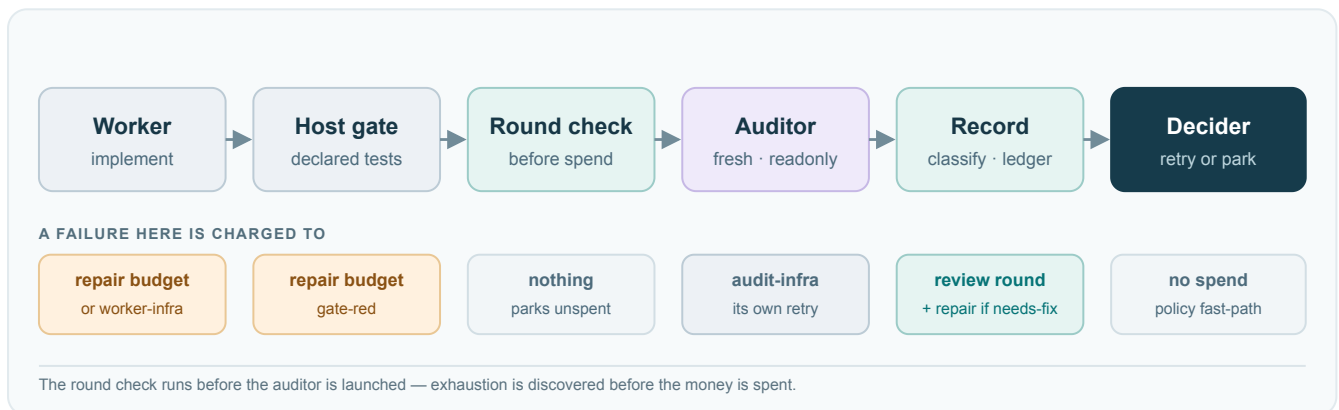


01 / START HERE

Two budgets meet at one line of Bash.

Review rounds bound how often a change may be judged. Repair passes bound how often it may be rewritten. They are different mechanisms with different keys — and one clamp couples them.

This report audits the shipped implementation against the live tree: `engine/review_policy.py` (1,191 lines), the driver `engine/l1-drive.sh`, `engine/task_lifecycle.py`, the frozen campaign policy `config-A16.json`, and the durable ledger at `.singular-state/review-policy/ledger.json` — nine logical changes, nineteen recorded rounds, four exceptions, and a 33-item non-blocking backlog. Every behavioural claim in sections 03 and 04 was reproduced by calling the live module, not inferred from reading it.



One attempt, six stations, three budget domains. Transport failures never reach the review ledger.

	REVIEW POLICY	WORKER REPAIR POLICY
Owner	<code>engine/review_policy.py</code>	<code>engine/l1-drive.sh</code>
Counts	completed, schema-valid auditor verdicts	implement → gate → audit passes
Keyed on	logical change — <code>dagNode</code> , else <code>taskId</code>	lease — <code>taskId</code>
Durable in	<code>review-policy/ledger.json</code>	the lease <code>retryCount</code>
Default	2 rounds, blocking P0 + P1	1 repair (normal) / 2 (high risk)
Survives the task	yes — spans successors on one node	no — per lease, reset by <code>unpark</code>

The clamp collapses the risk tier. After the tier resolves, the driver caps repairs at `maxReviewRounds - 1`. With the shipped ceiling of two rounds that is one repair — so a `Risk tier: high` task receives exactly what a normal task receives, and its risk-derived ceiling of two is unreachable. All eighteen `l1.review_policy_bound` events in this campaign carry the same pair: `maxReviewRounds 2`, `productRepairMax 1`.

02 / PROGRAMMABILITY

Everything is configurable. Not everything is equally privileged.

Both budgets resolve through a strict precedence chain. They do not resolve through the same one, and the difference is the point.

Review policy

```
code defaults
  2 rounds · [P0,P1] · classify required
↓
campaign config
  config-A16.json .reviewPolicy
  → lib.sh exports it as
    SINGULAR_REVIEW_POLICY_JSON
  → pinned in the campaign manifest
↓
repo config file
  $SINGULAR_JSON_CONFIG_FILE, else
  $SINGULAR_ROOT/singular.config.json
  (only if the env JSON is absent)
↓
per-field env
  SINGULAR_REVIEW_MAX_ROUNDS
  SINGULAR_REVIEW_BLOCKING_SEVERITIES
  SINGULAR_REVIEW_REQUIRE_CLASSIFICATION
↓
CLI exception
  grant --rounds --reason
        --evidence --authority [--task]
```

The `effective` verb prints the resolved policy with a `sources` map naming the winning layer per field. That is what makes the policy auditable at dispatch time rather than after the fact.

Worker repair policy

```
operator env
  SINGULAR_TASK_RISK_TIER
↓
task contract
  "Risk tier:" header line
↓
environment default
  SINGULAR_DEFAULT_RISK_TIER
↓
repo config
  .retryPolicy.defaultRiskTier
↓
built-in
  "normal"

normal | low | ordinary → ceiling 1
high | critical         → ceiling 2
unrecognised           → ceiling 2
                        fails SAFE to
                        the strict path

SINGULAR_MAX_RETRIES may only LOWER the
risk-derived ceiling, never raise it.
```

The privilege asymmetry. The task contract is a first-class input to the repair budget and has **no input at all** to the review budget. An author changes the repair tier by editing one markdown line; raising review rounds requires a CLI grant carrying `--authority` and at least one hashed evidence file. The direction is right — and worth stating, because the two headers sit three lines apart in the same contract.

What is hardcoded

Four things are configurable from no layer, by design: the severity vocabulary `P0–P3`; the verdict vocabulary `accepted` / `needs-fix` / `blocked` / `needs-human`; the evidence rule compiled into `classify()` — a P0 or P1 must carry `trigger`, `impact` and `requirement`; and the clamp, a literal in the driver. A grant may never exceed `maxReviewRounds` in one exception. An invalid value at any layer is a hard exit 2 — the driver refuses to spend money on a run whose policy will not load, rather than falling back to a default.

03 / LEDGER AND STATE INTEGRITY

The ledger keys on the change, not on the task.

That single decision is why review history survived three task supersessions in this campaign — and it is also where the one real reset vector lives.

```
memory-lifecycle      status=accepted  rounds=3  exceptions=2
1 TASK-1105 needs-fix  (historical, backfilled)
2 TASK-1105 needs-fix  (historical, backfilled)
3 TASK-1117 accepted
exc-cc6aba1a1be9 +2 bound TASK-1116 authorized, never used
exc-68487d55c6f8 +2 bound TASK-1117 the migration grant
```

Live excerpt. Three task identities, one continuous budget carried by the node.

Writes are `fcntl`-locked and atomic; read verbs take a shared lock, create no state and never rewrite, so `updatedAt` keeps meaning “last recorded change”. Backfilled rounds carry `historical: true` and still count toward `used` — provenance without free budget. Exceptions are task-bound by default, so the two grants above could never combine: the migration ceremony the supervisor performed was enforced by the code, not merely observed by it.

Can a task reset its budget?

VECTOR	RESETS?	WHY
Rename the branch	No	The branch name appears in no key. Renaming or recreating a worker branch has zero ledger effect.
New task id	No	With a DAG node declared, the successor inherits the node's full history. Verified live: <code>invocation-envelope</code> holds both TASK-1115 rounds.
Drop the DAG node	Yes	<code>logical_change_id()</code> falls back to <code>taskId</code> when <code>dagNode</code> is blank. A fresh budget is minted under a new key, with no grant and no authority.
<code>unpark</code>	Partly	Resets <code>retryCount</code> and clears <code>productPassStarted</code> ; does not touch the review ledger. Re-entry buys worker passes, never review rounds.

The reset vector lives in the least-privileged file in the system. A task contract is ordinary markdown that any author edits. Deleting or retyping its `DAG node:` line moves the most durable counter in the engine. Everything else that can extend a budget demands an authority string and hashed evidence.

Acceptance closes the candidate — and zeroes the count

`_open_round_count()` resets `used` to zero at every accepted verdict, so a later dispatch of the same node starts fresh at two. The intended “acceptance closes the candidate” rule, visible in the live ledger:

```
maintenance:a12-engine-boundary  needs-fix → accepted → needs-fix
                                   used 1 · allowed 2 · status open
```

The consequence: the ceiling is **per candidate, not per node**. A node that alternates accept and needs-fix has no cumulative bound. Defensible — each acceptance is a real gate passage — but not what “max 2 rounds” reads like in a config file.

04 / CORRECTNESS

Where it holds, and the three traps.

The accounting is honest and the gates fire in the right order. The failures that remain are all in one place: what happens when a verdict is malformed rather than wrong.

5

rounds recorded in the Claude phase — one per real verdict. No timeout, session-limit kill or provider failure produced a round.

33

P2/P3 findings diverted to `backlog.ndjson` instead of blocking a merge, across nine logical changes.

0

severity downgrades fired in the whole campaign. Trap 1 below is latent, not historical.

Working exactly as intended

The pre-audit `check` is the best design decision in the module: exhaustion is discovered before a paid call. Fail-closed classification works — a `needs-fix` carrying only prose findings synthesises `unclassified-N` items that are **blocking**, so an auditor cannot escape a fix by skipping the taxonomy. Every round stamps the reviewer's runner, model and effort, which made this campaign's provenance claims checkable afterwards.

TRAP 01

A genuine P0 can be downgraded into an acceptance

`classify()` requires `trigger`, `impact` and `requirement` to be non-blank on any P0 or P1, and demotes to P2 otherwise. The v1 schema requires only `id`, `severity` and `summary` — the three supporting fields are optional. So a **schema-valid** verdict exists in which the auditor reports a real P0 and the host converts the run to accepted. Reproduced against the live module:

```
input : verdict = needs-fix
       F1 severity=P0 summary + trigger + impact (no requirement)
output: effectiveVerdict = accepted
       downgraded = [F1]          applied = true
```

Nothing blocks, the round records as accepted, integration proceeds. The rule is defensible as an anti-inflation measure — but “the claim is unsupported” and “the verdict is now accepted” should not be the same branch.

TRAP 02

Legacy verdicts validate in warn mode

`SINGULAR_AUDIT_VERDICT_VALIDATE` defaults to `warn`. A v1 verdict is always fail-closed, but a v0 or legacy-schema verdict that fails validation emits `l1.audit_verdict_warned` and **proceeds to be recorded and to influence acceptance**. Campaign A16 runs the v1 contract, so it was never exposed. A consumer repository still on v0 is.

TRAP 03

Schema failures never reach the review budget at all

A packet or verdict that cannot be parsed is an infrastructure class, so it consumes worker-repair budget while the review ledger stays untouched. That is the TASK-1114 failure in one sentence: a complete candidate — later audited clean and integrated — spent its entire product budget on two missing `createdAt` fields and parked, **having never been reviewed once**. The 0.22.0 repair pass and the 0.23.0 `createdAt` default close the two known shapes; the structural gap, a format slip billed to the product budget, is still open.

05 / RECOMMENDATIONS

Three minimal changes, each independent of the others.

None requires new durable state, a new verb, or a migration. Two are single-function edits; the third is a classification move.

01

Never let a downgrade manufacture an acceptance

In `classify()`, when `downgraded` is non-empty and no blocking item remains, set `effectiveVerdict` to `needs-human` — already a valid verdict in the vocabulary — instead of `accepted`. Roughly four lines, no schema change, no new state.

Alternatively, or in addition: make those three fields `required` in the v1 schema for P0 and P1 items. That moves the failure into the auditor's own validation-repair loop, where a malformed verdict already belongs.

02

Make the logical-change key non-forgable

Require a `DAG node:` header in any contract that declares a `Target branch:`, and refuse dispatch when it is absent rather than falling back to `taskId`. Where a successor legitimately needs a new node, `grant --authority` is already the mechanism for that. This closes the only budget-reset vector a task author can reach unaided.

03

Give a format slip its own budget

Classify `packet-invalid` and `worker-no-packet` on an *unchanged* candidate as an infrastructure domain with its own single retry, rather than charging the product-repair budget. The park rule added in 0.22.0 already grants one re-emit; this makes the accounting match the intent. It is the change that would have saved TASK-1114 outright.

Two smaller follow-ons. Flip `SINGULAR_AUDIT_VERDICT_VALIDATE` to `strict` by default, keeping `warn` as an explicit opt-out for v0 consumers. And decide the coupling in section 01 deliberately: if high-risk tasks are meant to get two repairs, `maxReviewRounds` must be 3, because today the clamp erases the distinction the risk tier exists to make.

What this audit does and does not establish

ESTABLISHED

The precedence chains, the ledger keying and the three traps are properties of the shipped code, reproduced by direct invocation on 2026-09-16.

NOT ESTABLISHED

That trap 01 has ever fired — zero downgrades are recorded. That the policy holds under concurrent dispatch: this campaign ran one driver at a time.

Sources. `engine/review_policy.py` · `engine/l1-drive.sh` (risk tier, the clamp, the repair loop) · `engine/lib.sh` (config → env projection, `singular_lease_unpark`) · `engine/task_lifecycle.py` · `config-A16.json` · `review-policy/ledger.json` and `backlog.ndjson` · `events.ndjson` · `audit-verdict.v0/v1.schema.json`. **Runtime note.** The A16 campaign runtime is frozen and carries the pre-0.23.2 campaign-drift check; any campaign adopting these recommendations must be built from a runtime at 0.23.2 or later.